Django 로그인 · 회원관리 가이드

앞 ModelForm CRUD 프로젝트(`crud` / `articles`)에 `accounts` 앱 추가. 위에서부터 순서대로 작성.

전체 구조

```
crud/
├── crud/
│   ├── settings.py        # 🖍️ accounts 등록, AUTH_USER_MODEL
│   └── urls.py            # 🖍️ accounts/ 추가
├── templates/
│   └── base.html          # 🖍️ 로그인 상태별 메뉴
├── articles/
│   └── views.py           # 🖍️ index에 @login_required
└── accounts/              # 🆕
    ├── models.py          # 커스텀 User
    ├── admin.py           # 관리자 페이지 등록
    ├── forms.py           # 가입 · 수정 폼
    ├── urls.py
    ├── views.py
    └── templates/accounts/
        ├── login.html
        ├── signup.html
        └── update.html
```

기능 한눈에 보기

기능	URL	method	view	핵심 도구
로그인	`accounts/login/`	GET, POST	`login`	`AuthenticationForm`, `auth_login()`
로그아웃	`accounts/logout/`	POST	`logout`	`auth_logout()`
회원가입	`accounts/signup/`	GET, POST	`signup`	`UserCreationForm` 상속
회원정보 수정	`accounts/update/`	GET, POST	`update`	`UserChangeForm` 상속
회원탈퇴	`accounts/delete/`	POST	`delete`	`request.user.delete()`

---

1. accounts 앱 생성 + 커스텀 User

1-1. 앱 생성

```
bash
```

```
python manage.py startapp accounts
```

- 앱 이름 `accounts` 권장 : `@login_required` 기본 이동 주소 `/accounts/login/`과 일치
- `account` 등 다른 이름 사용 시 `settings.py`에 `LOGIN_URL = 'account:login'` 추가 필요

1-2. 설정

`crud/settings.py`

```
python

INSTALLED_APPS = [
    'articles',
    'accounts',          # 앱 등록
    'django.contrib.admin',
    ...
]

# 맨 아래 추가
AUTH_USER_MODEL = 'accounts.User'
```

- `AUTH_USER_MODEL` : 기본 User 대신 우리 앱의 User 사용 선언
- **첫 migrate 전에** 설정 필수 (나중에 바꾸기 매우 어려움)

1-3. User 모델

`accounts/models.py`

```
python

from django.contrib.auth.models import AbstractUser


class User(AbstractUser):
    pass
```

- 기본 User와 기능 동일, 지금은 빈 클래스
- 이유 : 나중에 필드(닉네임, 프로필 등) 추가 가능하도록 미리 교체 (Django 공식 권장)

1-4. 관리자 페이지 등록

`accounts/admin.py`

```
python
```

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from .models import User

admin.site.register(User, UserAdmin)
```

- `UserAdmin` : 비밀번호 해시 처리 등 회원 전용 관리 화면

1-5. DB 반영

```
bash

# 기존 db.sqlite3 있으면 먼저 삭제
python manage.py makemigrations
python manage.py migrate
```

---

2. 관리자 계정 생성 (admin / 1234)

```
bash

python manage.py createsuperuser
```

```
Username: admin
Email address:                    ← 엔터 (생략 가능)
Password:                         ← 1234 (입력 내용 안 보임)
Password (again):                 ← 1234
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

- 경고 3줄 : 비밀번호 규칙 위반 안내 → `y` 입력 시 무시하고 생성 (실습용)
- 확인 : `http://127.0.0.1:8000/admin/` 로그인 → Users 목록에 `admin`

---

3. URL 작성

3-1. 프로젝트 URL

`crud/urls.py`

```
python
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('articles/', include('articles.urls')),
    path('accounts/', include('accounts.urls')),   # 추가
]
```

3-2. 앱 URL

`accounts/urls.py`

python

```python
from django.urls import path
from . import views

app_name = 'accounts'

urlpatterns = [
    path('login/', views.login, name='login'),      # 로그인 화면 + 로그인 요청
    path('logout/', views.logout, name='logout'),   # 로그아웃 요청
    path('signup/', views.signup, name='signup'),   # 회원가입 화면 + 가입 요청
    path('update/', views.update, name='update'),   # 회원정보 수정 화면 + 수정 요청
    path('delete/', views.delete, name='delete'),   # 회원탈퇴 요청
]
```

- `<int:pk>` 없음 : 대상이 항상 로그인한 본인(`request.user`) → 주소로 받을 필요 없음

3-3. 공통 메뉴 (로그인 상태별 분기)

`templates/base.html`

html

```
<body>
  <header>
    {% if user.is_authenticated %}
      <b>{{ user.username }}</b>님 |
      <a href="{% url "articles:index" %}">홈</a> |
      <a href="{% url "articles:create" %}">새 글</a> |
      <a href="{% url "accounts:update" %}">회원정보 수정</a>
      <form action="{% url "accounts:logout" %}" method="POST" style="display:inline;">
        {% csrf_token %}
        <input type="submit" value="로그아웃">
      </form>
    {% else %}
      <a href="{% url "accounts:login" %}">로그인</a> |
      <a href="{% url "accounts:signup" %}">회원가입</a>
    {% endif %}
  </header>
  {% block content %}{% endblock content %}
</body>
```

항목	의미
`user`	현재 사용자. view에서 안 넘겨도 모든 템플릿에서 사용 가능
`user.is_authenticated`	로그인 상태면 True
로그아웃이 `<form>`인 이유	세션 삭제 = 데이터 변경 → POST

---

4. views.py 상단 import

`accounts/views.py`

```python
from django.shortcuts import render, redirect
from django.contrib.auth import login as auth_login
from django.contrib.auth import logout as auth_logout
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required
from .forms import CustomUserCreationForm, CustomUserChangeForm
```

- `as auth_login` : 우리 view 함수 이름 `login`과 충돌 방지 (logout도 동일)
- `.forms` import는 6·7단계에서 forms.py 작성 후 추가

---

5. 로그인

5-1. view

python

```python
def login(request):
    if request.user.is_authenticated:          # 이미 로그인 → 목록으로
        return redirect('articles:index')

    if request.method == 'POST':
        form = AuthenticationForm(request, request.POST)
        if form.is_valid():                    # 아이디·비밀번호 일치 확인
            auth_login(request, form.get_user())
            return redirect(request.GET.get('next') or 'articles:index')
    else:
        form = AuthenticationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)
```

코드	의미
`AuthenticationForm`	로그인 전용 폼 (ModelForm 아님, DB 저장 X)
`AuthenticationForm(request, request.POST)`	첫 인자 `request` 필수 (ModelForm과 차이)
`form.get_user()`	인증 성공한 User 객체
`auth_login(request, user)`	세션 생성 → 로그인 상태 유지
`request.GET.get('next')`	로그인 전 가려던 주소 있으면 그쪽으로

5-2. 템플릿

`accounts/templates/accounts/login.html`

html

```
{% extends "base.html" %}
{% block content %}
<h1>로그인</h1>
<form action="" method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <input type="submit" value="로그인">
</form>
{% endblock content %}
```

- `action=""` : 현재 주소 그대로 전송 → `?next=...` 유지

5-3. 로그인했을 때만 목록 보여주기

`articles/views.py`

```
python

from django.contrib.auth.decorators import login_required

@login_required
def index(request):
    ...
```

비로그인 상태에서 목록 접근 시 흐름

순서	동작
1	`/articles/` 요청
2	`@login_required` → `/accounts/login/?next=/articles/` 로 이동
3	로그인 성공 → `next` 값 `/articles/` 로 이동

- 다른 view(`create`, `update` 등)도 같은 방식으로 보호 가능

6. 로그아웃

```
python

def logout(request):
    if request.method == 'POST':
        auth_logout(request)
    return redirect('accounts:login')
```

- `auth_logout(request)` : 세션 삭제
- GET 요청(주소창 직접 입력)은 무시 → 로그인 화면으로
- 템플릿 없음 : 버튼은 `base.html`에 위치

---

7. 회원가입

7-1. 폼

`accounts/forms.py`

```python
from django.contrib.auth import get_user_model
from django.contrib.auth.forms import UserCreationForm, UserChangeForm


class CustomUserCreationForm(UserCreationForm):
    class Meta(UserCreationForm.Meta):
        model = get_user_model()
```

- 상속 이유 : 기본 `UserCreationForm`은 기본 User 기준 → 커스텀 User에 그대로 사용 시 에러
- `get_user_model()` : 현재 활성 User(`accounts.User`) 반환
- `Meta(UserCreationForm.Meta)` : 기존 설정(필드 등) 그대로 상속

7-2. view

```python
def signup(request):
    if request.user.is_authenticated:
        return redirect('articles:index')

    if request.method == 'POST':
        form = CustomUserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            auth_login(request, user)          # 가입 후 바로 로그인
            return redirect('articles:index')
    else:
        form = CustomUserCreationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/signup.html', context)
```

- 구조 : CRUD의 `create`와 동일

- `form.save()` : 비밀번호 암호화(해시)까지 자동 처리 후 저장
- 입력 필드 : `username`, `password1`, `password2` (확인용)

7-3. 템플릿

`accounts/templates/accounts/signup.html`

```
html

{% extends "base.html" %}
{% block content %}
<h1>회원가입</h1>
<form action="" method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <input type="submit" value="가입하기">
</form>
{% endblock content %}
```

---

8. 회원정보 수정

8-1. 폼

`accounts/forms.py` 에 추가

```
python

class CustomUserChangeForm(UserChangeForm):
    password = None   # 비밀번호 안내 문구 숨김

    class Meta(UserChangeForm.Meta):
        model = get_user_model()
        fields = ('email', 'first_name', 'last_name',)
```

코드	의미
`fields` 지정	기본값은 전체 필드 → `is_superuser` 등 권한 필드까지 노출되므로 제한
`password = None`	기본 "비밀번호 변경 링크" 문구 제거 (이 가이드에서 비밀번호 변경 미구현)

8-2. view

```
python
```

```python
@login_required
def update(request):
    if request.method == 'POST':
        form = CustomUserChangeForm(request.POST, instance=request.user)
        if form.is_valid():
            form.save()
            return redirect('articles:index')
    else:
        form = CustomUserChangeForm(instance=request.user)
    context = {
        'form': form,
    }
    return render(request, 'accounts/update.html', context)
```

- 구조 : CRUD의 `update`와 동일
- 차이 : `Article.objects.get(pk=pk)` 대신 `request.user`
- `@login_required` : 비로그인 사용자 접근 차단

8-3. 템플릿 (탈퇴 버튼 포함)

`accounts/templates/accounts/update.html`

```html
{% extends "base.html" %}
{% block content %}
<h1>회원정보 수정</h1>
<form action="" method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <input type="submit" value="수정">
</form>
<hr>
<form action="{% url "accounts:delete" %}" method="POST">
  {% csrf_token %}
  <input type="submit" value="회원탈퇴">
</form>
{% endblock content %}
```

---

9. 회원탈퇴

```python
```

```
def delete(request):
    if request.method == 'POST' and request.user.is_authenticated:
        request.user.delete()
        auth_logout(request)
    return redirect('accounts:login')
```

- 순서 중요 : **탈퇴 → 로그아웃**
- 반대로 하면 로그아웃 시점에 `request.user`가 비로그인 사용자로 바뀌어 삭제 불가
- 구조 : CRUD의 `delete`와 동일 (POST일 때만 삭제)

---

10. 실행 확인

```
bash

python manage.py runserver
```

#	동작	기대 결과
1	로그아웃 상태로 `/articles/` 접속	`/accounts/login/?next=/articles/` 로 이동
2	`admin` / `1234` 로그인	목록 표시, 상단에 `admin님` + 로그아웃 버튼
3	틀린 비밀번호 로그인	로그인 화면 + 에러 메시지
4	로그아웃 클릭	로그인 화면, `/articles/` 접근 시 다시 로그인 요구
5	회원가입	가입 즉시 로그인 상태로 목록 이동
6	회원정보 수정 (이메일·이름)	저장 후 목록 이동, `/admin/` 에서 변경 확인
7	회원탈퇴	로그인 화면 이동, `/admin/` Users 목록에서 사라짐

예정된 막힘

증상	원인	대처
`InconsistentMigrationHistory`	기본 User로 migrate한 DB에 커스텀 User 적용	`db.sqlite3` 삭제 → `migrate` 다시
`Manager isn't available; 'auth.User' has been swapped`	기본 `UserCreationForm` 그대로 사용	`CustomUserCreationForm` 사용
목록 접속 시 `/accounts/login/` 404	앱 이름·URL을 `account`로 생성	`LOGIN_URL = 'account:login'` 설정
`login() takes 1 positional argument but 2 were given`	`auth_login` 별칭 없이 import → view 이름과 충돌	`from django.contrib.auth import login as auth_login`
로그아웃 눌러도 로그인 유지	`<a>` 링크(GET)로 요청	`<form method="POST">` 사용
수정 화면에 `is_superuser` 등 노출	`CustomUserChangeForm`에 `fields` 미지정	`fields = ('email', 'first_name', 'last_name',)`
탈퇴 안 됨	`auth_logout` 먼저 호출	`delete()` → `auth_logout()` 순서
폼 라벨·에러가 영어	`LANGUAGE_CODE = 'en-us'` (기본값)	`settings.py`에서 `'ko-kr'`로 변경
urls.py 작성 직후 `AttributeError: module 'accounts.views' has no attribute ...`	아직 없는 view 함수 연결	해당 view 먼저 작성하거나 그 줄 주석 처리